

第四次编程实践课

软件工程与计算 I · 2026 · 南京大学

| Week 4 · 模块导入 · 数组与 ArrayList · DevOps 工具链

今日安排

知识讲解

Java 模块导入 · 数组 · String · ArrayList

import · 包 · 数组操作 · 字符串方法 · 动态列表

AI实践 DevOps 开发工具链

Git 版本控制 · CI/CD 自动化 · 现代开发工作流

大作业 02-Lexer 答案解析 · 03-AST 布置

词法分析器参考答案点评 · 语法分析器任务讲解

当堂练习

练习 Lab04 · 共 4 题，独立完成

- 练习 1-2: 数组基础操作与 ArrayList 入门 (中等)
- 练习 3-4: 数组与 ArrayList 综合 (较难)

目标: 熟练掌握数组遍历与变换、ArrayList 增删改查, 建立"动态 vs 静态"集合的直觉

PART 1 · Java 知识讲解

模块导入 · 数组 · String · ArrayList

import 与包 (Package)

包 = Java 的文件夹组织方式，避免类名冲突，控制访问范围。

```
package edu.nju.software.token;    // 声明当前类属于哪个包

import java.util.ArrayList;        // 导入单个类
import java.util.Scanner;          // 导入单个类
import java.util.*;                // 导入整个包（通配符）
```

自动导入

`java.lang.*` 包自动导入，无需手写——`String`、`Math`、`System`、`Integer` 等都在此包中。

同名类冲突

```
import java.util.Date;
import java.sql.Date;           // 编译错误！两个 Date 冲突

// 解决：一个 import，另一个写全限定名
import java.util.Date;
java.sql.Date sqlDate = new java.sql.Date(System.currentTimeMillis());
```

静态导入与访问控制

静态导入 (Static Import)

```
import static java.lang.Math.PI;  
import static java.lang.Math.sqrt;  
  
double r = sqrt(PI);    // 不需要写 Math.sqrt(Math.PI)
```

访问控制与包的关系

修饰符	同类	同包	子类	其他包
public	Y	Y	Y	Y
protected	Y	Y	Y	N
默认 (不写)	Y	Y	N	N
private	Y	N	N	N

规律：不 public 的类，其他包无法 import。大作业中 Token 类都是 public，所以 Lexer 可以跨包使用。

常用包速查

包	常用类	场景
java.lang.*	String, Math, Integer, System	自动导入，无需写
java.util.*	ArrayList, Scanner, Arrays, HashMap	集合、输入、工具
java.io.*	File, BufferedReader, IOException	文件读写
java.time.*	LocalDate, LocalTime	日期时间 (Java 8+)

大作业中的导入

```
// Lexer.java 中你会用到的导入
import edu.nju.software.stream.CharLookaheadStream; // 字符流
import edu.nju.software.token.*;                    // 所有 Token 类型
import java.util.ArrayList;                          // 动态列表
```

经验：IDE (IntelliJ) 会自动提示缺失的 import，按 `Alt+Enter` 即可自动添加。

数组（Array） — 声明与访问

数组 = 固定长度的连续存储容器，声明时必须指定长度或初始值。

```
// 两种声明方式
int[] scores = new int[5];           // 长度固定为 5，默认值 0
int[] scores = {85, 42, 90, 58, 73}; // 用字面量初始化

// 访问与修改
scores[0] = 100;                      // 修改第一个元素
int first = scores[0];                // 读取：100
int len   = scores.length;            // 长度：5（注意：不是方法，没有括号）
```

越界访问 `scores[5]` → `ArrayIndexOutOfBoundsException`

`arr.length` 是属性（没括号），而 `list.size()` 是方法（有括号）——常见混淆点。

数组（Array） — 遍历

方式 1: 传统 for（需要下标时）

```
for (int i = 0; i < scores.length; i++) {  
    System.out.println("第" + i + "个: " + scores[i]);  
}
```

方式 2: 增强 for-each（只需要值时，更简洁）

```
for (int s : scores) {  
    System.out.println(s);  
}
```

	传统 for	增强 for-each
能拿到下标	<code>i</code> 可用	拿不到
能修改元素	<code>arr[i] = x</code>	不能
语法简洁度	较长	更短

选择建议： 需要下标或修改元素时用传统 for，只读遍历用 for-each。

数组 — 常见操作模式(1/2)

1. 累加求和（累加器模式）

```
int sum = 0;
for (int s : scores) {
    sum += s;    // 每轮把当前元素加到 sum
}
```

累加器模式是数组操作中最基础的模式：初始化变量 → 遍历累加 → 返回结果。

变体：求最大值（`max`）、计数（`count++`）、拼接字符串（`StringBuilder.append`）。

2. 线性查找

```
static int indexOf(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i;    // 找到，立即返回下标
    }
    return -1;    // 遍历完仍未找到，返回 -1
}
```

约定：查找失败返回 `-1`（Java 标准库也用这个约定，如 `String.indexOf`）。

数组 — 常见操作模式(2/2)

3. 生成新数组（反转）

```
static int[] reverse(int[] arr) {  
    int[] result = new int[arr.length];    // 必须 new 新数组  
    for (int i = 0; i < arr.length; i++) {  
        result[i] = arr[arr.length - 1 - i];  
    }  
    return result;    // 返回新数组，原数组不变  
}
```

4. 截取子数组

```
static int[] copyRange(int[] arr, int from, int to) {  
    int[] result = new int[to - from];  
    for (int i = 0; i < result.length; i++) {  
        result[i] = arr[from + i];  
    }  
    return result;  
}
```

核心：数组长度不可变，需要"变长"时要么 new 新数组，要么用 ArrayList。

String 常用方法(1/2)

String 是不可变对象——每次"修改"都会创建新字符串。

方法	功能	示例
s.length()	长度	"hello".length() → 5
s.charAt(i)	第 i 个字符	"hello".charAt(0) → 'h'
s.equals(other)	内容比较	"ab".equals("ab") → true
s.contains(sub)	是否包含子串	"hello".contains("ell") → true
s.split(sep)	分割为数组	"a b c".split(" ") → ["a","b","c"]
s.substring(i,j)	截取 [i, j)	"hello".substring(1,3) → "el"
s.isEmpty()	是否为空	"".isEmpty() → true
s.indexOf(sub)	子串位置	"hello".indexOf("ll") → 2
s.toUpperCase()	转大写	"hi".toUpperCase() → "HI"

s.length() 是方法（有括号）， arr.length 是属性（没括号）——常见混淆点。

String — 陷阱: `==` vs `.equals()`

```
String a = new String("hello");
String b = new String("hello");
System.out.println(a == b);      // false (比较引用地址)
System.out.println(a.equals(b)); // true (比较内容)
```

规则: 比较字符串内容永远用 `.equals()`, 不用 `==`。

为什么 `==` 会出错?

- `==` 比较的是两个变量是否指向**同一个对象** (内存地址)
- `.equals()` 比较的是两个对象的**内容**是否相同
- `new String("hello")` 每次创建新对象 → 地址不同 → `==` 为 `false`

这个陷阱在 `ArrayList<String>` 的 `.contains()` 中不会出现——`.contains()` 内部用的是 `.equals()`。

String — 字符串拼接三种方式

```
// 方式 1: + 运算符 (少量拼接, 简洁)
String msg = "Hello, " + name + "!";

// 方式 2: String.format (格式化输出, 适合混合类型)
String msg = String.format("最高分:%d 平均分:%.1f", max, avg);

// 方式 3: StringBuilder (大量拼接, 性能最好)
StringBuilder sb = new StringBuilder();
for (String w : words) { sb.append(w).append(" "); }
String result = sb.toString();
```

方式	适用场景	大作业中的使用
+	拼接 2-3 次	<code>countTokens</code> 的返回值
<code>String.format</code>	格式化数字	Lab 练习中的报告生成
<code>StringBuilder</code>	循环拼接	<code>tokensToString</code>

ArrayList — 动态列表

ArrayList = 长度可变的数组，来自 `java.util.ArrayList`。

```
import java.util.ArrayList;

ArrayList<Integer> list = new ArrayList<>(); // 泛型：只能放 Integer
list.add(10); // 添加元素
list.add(20);
list.add(30);
list.get(0); // 获取：10
list.set(1, 25); // 修改：第1个变为 25
list.remove(0); // 删除第0个（后面的元素前移）
list.size(); // 当前长度：2
list.contains(25); // 是否包含：true
```

自动装箱 / 拆箱

```
list.add(42); // int 42 自动装箱为 Integer.valueOf(42)
int val = list.get(0); // Integer 自动拆箱为 int
```

`ArrayList<int>` 不合法！泛型只能用包装类：`Integer`、`Double`、`String`。

数组 vs ArrayList 对比

特性	数组 <code>int[]</code>	ArrayList <code>ArrayList<Integer></code>
长度	固定	可变
语法	<code>arr[i]</code>	<code>list.get(i)</code>
存放基本类型	可以	不可以（需包装类）
性能	更快（无装箱）	略慢（装箱开销）
适合场景	长度已知	长度未知 / 需要增删

互转

```
// 数组 → ArrayList
ArrayList<Integer> list = new ArrayList<>();
for (int n : arr) list.add(n);

// ArrayList → 数组
int[] arr = new int[list.size()];
for (int i = 0; i < list.size(); i++) arr[i] = list.get(i);
```

实践选择：不确定元素个数时先用 ArrayList 收集，最后按需转数组。

ArrayList 删除元素的陷阱(1/2)

正向遍历删除 — 会跳过元素!

```
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(3, 3, 5, 3, 7));  
// 目标: 删除所有值为 3 的元素  
  
for (int i = 0; i < list.size(); i++) {  
    if (list.get(i) == 3) {  
        list.remove(i);    // 删除后, 后面的元素前移  
        // i++ 跳到下一个位置, 但原来 i+1 的元素已经移到 i 了!  
    }  
}  
  
// 结果: [3, 5, 7] ← 第二个 3 被跳过了!
```

删除前: [3, 3, 5, 3, 7] i=0, 发现3, 删除

↓

删除后: [3, 5, 3, 7] i=1 (跳过了新的第0个元素 3!)

根因: 删除使元素前移, 而 `i++` 仍然向后走, 导致"漏网之鱼"。

ArrayList 删除元素的陷阱(2/2)

解决方案：反向遍历

```
for (int i = list.size() - 1; i >= 0; i--) {  
    if (list.get(i) == 3) {  
        list.remove(i);    // 反向删除，前面的下标受影响  
    }  
}  
// 结果: [5, 7] ✓
```

反向遍历时，删除的元素在已遍历区域，不会影响待遍历的部分。

另一个陷阱：remove(int) vs remove(Object)

```
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(10, 20, 30));  
  
list.remove(1);                // 删除下标为 1 的元素 → [10, 30]  
list.remove(Integer.valueOf(30)); // 删除值为 30 的元素 → [10]
```

传 `int` 删的是下标，传 `Integer` 删的是值——务必注意区分！

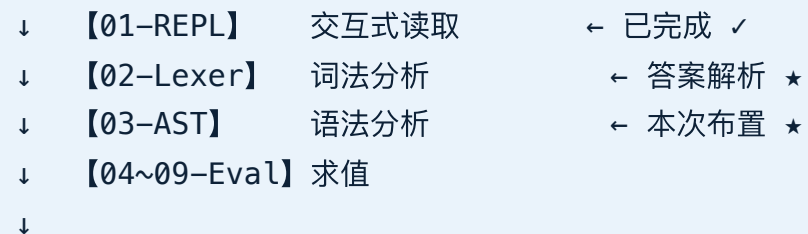
PART 2 · 大作业讲解

Project 02 Lexer 答案解析 · Project 03 AST 布置

大作业进度回顾

解释器流水线

用户输入字符串



输出结果

每一层都建立在前一层之上：Lexer 生成 Token 序列，AST 把 Token 组织成树。

Project 02 Lexer — 答案解析：整体架构

文件结构与三个方法

Lexer.java

- └─ isSpecialCharacter(char) ← 私有辅助方法
- └─ lex(CharLookaheadStream) ← 核心：字符 → Token 列表
- └─ countTokens(ArrayList<Token>) ← 扩展1：分类计数
- └─ tokensToString(ArrayList<Token>) ← 扩展2：Token → 字符串

注意 `LeftParen.create()` 是工厂方法（而非 `new LeftParen()`），这是 Token 类的设计细节。

Project 02 Lexer — 答案解析: `lex()` 主循环

字符级状态机

```
while (!input.eof()) {  
    char c = input.consume();           // 先消耗一个字符  
    if (Character.isWhitespace(c)) continue; // 空白 → 跳过  
    switch (c) {  
        case '(': tokens.add(LeftParen.create()); break;  
        case ')': tokens.add(RightParen.create()); break;  
        case '"': // → 进入字符串解析模式  
        default: // → 判断是整数还是符号  
    }  
}
```

设计要点:

- 一个 `while` 循环 + `switch` 分支, 每个分支职责单一
- 括号是最简单的: 消耗一个字符, 直接创建 Token
- `"` 和 `default` 分支较复杂, 接下来逐一讲解

Project 02 Lexer — 答案解析: `isSpecialCharacter`

辅助方法：判断"特殊字符"

```
private static boolean isSpecialCharacter(char c) {  
    return c == '(' || c == ')' || c == '"' || Character.isWhitespace(c);  
}
```

为什么需要这个方法?

- 读取符号（如 `+`、`foo`、`let*`）时，需要知道"什么时候停下来"
- 遇到括号、引号、空白 → 当前符号结束，不再继续读
- 提取为方法 → 避免重复写四个条件，名字即文档

Project 02 Lexer — 答案解析: `isSpecialCharacter` 调用

它在符号读取循环中被调用

```
// 符号读取循环:
StringBuilder builder = new StringBuilder();
builder.append(c); // 首字符
while (!input.eof()) {
    c = input.lookahead(1);
    if (isSpecialCharacter(c)) break; // ← 遇到特殊字符就停
    builder.append(c);
    input.consume();
}
tokens.add(SymbolToken.create(builder.toString()));
```

如果没有这个方法会怎样?

```
// ✗ 没有 isSpecialCharacter: 符号会"吃掉"括号和引号
// 输入 "foo(" → 期望 [SymbolToken(foo), LeftParen]
//           → 实际 [SymbolToken(foo())] ← 括号被吞了!
```

设计原则: 提取辅助方法 = 给判断逻辑取名字, 让代码自文档化。

Project 02 Lexer — 答案解析：负整数判断

lookahead 的经典应用

```
if (Character.isDigit(c) ||  
    (c == '-' && !input.eof() && Character.isDigit(input.lookahead(1)))) {  
    // 进入整数读取模式  
}
```

为什么需要 lookahead(1) ?

- 读到 `-` 时，不知道是"减号"还是"负号"
- 看下一个字符：是数字 → 负整数开头（如 `-42`）；不是 → 普通符号（如 `-`）
- 三个条件缺一不可：`c == '-'` + `!input.eof()` + 下一个是数字

这个判断是学生出错最多的地方——漏掉 `!input.eof()` 会在字符串末尾的 `-` 上崩溃。

Project 02 Lexer — 答案解析：整数贪心读取

lookahead → 确认 → **consume** 循环模式

```
StringBuilder builder = new StringBuilder();
builder.append(c);                                // 先把首字符放入 (数字或 '-')
while (!input.eof()) {
    c = input.lookahead(1);                        // 预览下一个, 不消耗
    if (!Character.isDigit(c)) break;              // 不是数字 → 整数结束
    builder.append(c);
    input.consume();                               // 确认是数字, 才消耗
}
tokens.add(IntegerToken.create(Integer.parseInt(builder.toString())));
```

关键 API

- **Integer.parseInt("-42")** : 将字符串 **"-42"** 转为 **int** 值 **-42**
- **IntegerToken.create(n)** : 工厂方法, 创建一个存储整数值的 Token

贪心模式: **lookahead** 判断 → 确认后 **consume** → 循环, 直到不匹配——符号读取也用同样的模式。

Project 02 Lexer — 答案解析：符号读取

符号贪心读取循环（与整数对称）

```
// default 分支中，不是数字/负号 → 当作符号处理
StringBuilder builder = new StringBuilder();
builder.append(c); // 首字符
while (!input.eof()) {
    c = input.lookahead(1); // 预览
    if (isSpecialCharacter(c)) break; // 遇到特殊字符 → 停止
    builder.append(c);
    input.consume(); // 确认后消耗
}
tokens.add(SymbolToken.create(builder.toString()));
```

整数 vs 符号——结构完全对称

	整数读取	符号读取
首字符	数字 或 <code>-</code> +数字	其他任意字符
继续条件	<code>Character.isDigit(c)</code>	<code>!isSpecialCharacter(c)</code>
停止条件	遇到非数字	遇到 <code>(</code> <code>)</code> <code>"</code> 空白
创建 Token	<code>IntegerToken.create(Integer.parseInt(...))</code>	<code>SymbolToken.create(...)</code>

Project 02 Lexer — 答案解析：字符串解析

escape 状态标志

```
case '': {
    StringBuilder builder = new StringBuilder();
    boolean terminate = false;
    boolean escape = false;
    while (!input.eof()) {
        c = input.consume();
        if (!escape && c == '') { terminate = true; break; }
        builder.append(c);
        if (escape) escape = false;
        else if (c == '\\') escape = true;
    }
}
```

escape 标志的工作流程：

- 读到 `\` → 设 `escape = true`
- 下一个字符（如 `"`）→ 不触发结束，正常追加，恢复 `escape = false`
- 效果： `\"` 被当作普通字符，不会结束字符串

Project 02 Lexer — 答案解析：字符串细节

未终止检测 + 第三方反转义

```
if (!terminate) throw new RuntimeException("Unterminated string");
tokens.add(StringToken.create(
    StringEscapeUtils.unescapeJava(builder.toString())));
}
```

关键细节

- **terminate 标志**：循环结束时仍为 **false** → 字符串没有闭合引号 → 抛异常（防御性编程）
- **StringEscapeUtils.unescapeJava()**：来自 Apache Commons Text 库
 - **\\n** → 真正的换行符 **\n**
 - **\\\"** → 双引号 **"**
 - **** → 反斜杠 ****
- 这是**第三方库调用**——**pom.xml** 中已配置依赖，你不需要自己实现反转义

Project 02 Lexer — 答案解析: `countTokens`

`instanceof` 分类计数

```
public static String countTokens(ArrayList<Token> tokens) {  
    int integers = 0, symbols = 0, strings = 0, leftParens = 0, rightParens = 0;  
    for (Token t : tokens) {  
        if (t instanceof IntegerToken) integers++;  
        else if (t instanceof SymbolToken) symbols++;  
        else if (t instanceof StringToken) strings++;  
        else if (t instanceof LeftParen) leftParens++;  
        else if (t instanceof RightParen) rightParens++;  
    }  
    return "整数:" + integers + " 符号:" + symbols + " 字符串:" + strings  
        + " 左括号:" + leftParens + " 右括号:" + rightParens;  
}
```

设计细节

- `instanceof`：判断对象的实际类型，后续学习"多态"时会深入讨论
- 字符串拼接用 `+`：这里拼接次数少，直接用 `+` 简洁易读（对比 `tokensToString` 用 `StringBuilder`）
- 格式严格：冒号是半角 `:`，不是全角 `:` ——一个字符之差就导致测试失败

`instanceof` 的本质：父类变量 `Token t` 可以指向子类对象 `IntegerToken`，用 `instanceof` 判断实际是哪个子类。

Project 02 Lexer — 答案解析: `tokensToString` (1/2)

整体结构: 遍历 + 分支转换

```
public static String tokensToString(ArrayList<Token> tokens) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < tokens.size(); i++) {
        if (i > 0) sb.append(" ");          // Token 之间加空格 (首个不加)
        Token t = tokens.get(i);
        if (t instanceof IntegerToken) {
            sb.append(((IntegerToken) t).value);
        } else if (t instanceof StringToken) {
            sb.append("\"").append(((StringToken) t).value).append("\"");
        } else if (t instanceof SymbolToken) {
            sb.append(((SymbolToken) t).name);
        }
    }
}
```

`StringBuilder`: 循环中拼接多次, 用 `StringBuilder` 比 `+` 性能更好。

Project 02 Lexer — 答案解析: `tokensToString` (2/2)

续: 括号分支 + 强制转型解析

```
        } else if (t instanceof LeftParen) {
            sb.append("(");
        } else if (t instanceof RightParen) {
            sb.append(")");
        }
    }
    return sb.toString();
}
```

强制转型: 为什么需要 `((IntegerToken) t).value`?

```
Token t = tokens.get(i);           // 编译器只知道 t 是 Token
t.value;                           // ❌ 编译错误! Token 没有 .value
((IntegerToken) t).value;          // ✅ 先转型为子类, 再访问子类字段
```

步骤: 先用 `instanceof` 确认类型 → 再用 `(SubType)` 强制转型 → 最后访问子类字段。
这是 Java 多态的基础操作, 后续课程会深入讲解。

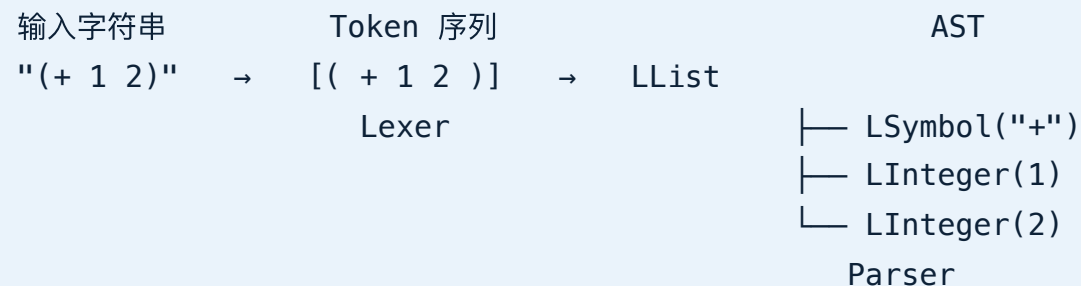
Project 02 — 常见错误总结

错误	原因	修正
负整数被拆为 - + 数字	没有用 lookahead 判断	先看下一个字符是否为数字
字符串中 \" 导致提前截断	没有维护 escape 状态	加 escape 标志
未闭合字符串没报错	没检查 terminate	加 if (!terminate) throw ...
countTokens 格式错误	冒号全角/半角不对	严格匹配：半角 : 不是 ：
符号读取不停	没判断 (,) , \" , 空白	用 isSpecialCharacter() 终止
NullPointerException	eof() 后仍调用 consume()	循环条件加 !input.eof()

教训：词法分析器的"魔鬼在细节"——每个字符的边界条件都要考虑。

Project 03 — AST 语法分析器

语法分析 (Parsing): 将 Token 序列组织为**抽象语法树 (AST)**。



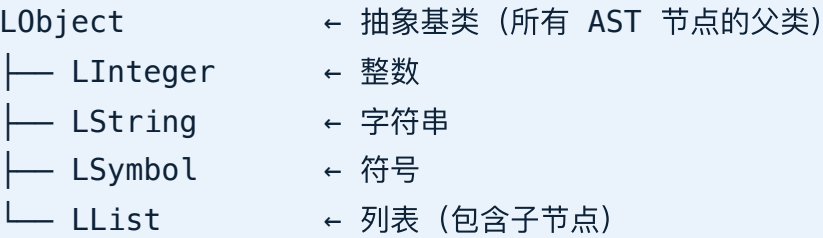
为什么需要 AST?

- Token 序列是**扁平的**: `[( + 1 2 )]`, 看不出层次
- AST 是**树形的**: 表达了"用 + 对 1 和 2 求和"的结构
- 后续的求值器 (Eval) 基于 AST 递归计算

Project 03 — AST 节点类型

类	字段	含义	示例
LInteger	int value	整数字面量	LInteger(42)
LString	String value	字符串字面量	LString("hello")
LSymbol	String name	符号	LSymbol("+")
LList	List<LObject> 通过 getContent()	列表	LList[+, 1, 2]

类层次



这里用到了**继承**和**多态**——下周的课程主题。现在先会用即可。

Project 03 — TokenReadStream 接口

TokenReadStream 是 Token 流工具，类似 Lexer 中的 CharLookaheadStream：

方法	作用	说明
peek()	预览当前 Token， 不移动 位置	到末尾返回 <code>null</code>
next()	消耗并返回当前 Token，位置 后移	

与 CharLookaheadStream 的对比

	CharLookaheadStream	TokenReadStream
操作对象	字符 <code>char</code>	Token 对象
预览方法	<code>lookahead(1)</code>	<code>peek()</code>
消耗方法	<code>consume()</code>	<code>next()</code>
末尾标记	<code>eof()</code> 返回 <code>true</code>	<code>peek()</code> 返回 <code>null</code>

同一模式：预览 → 判断 → 消耗。Parser 使用 TokenReadStream 就像 Lexer 使用 CharLookaheadStream。

Project 03 — 任务零：实现 TokenReadStream

```
public class TokenReadStream {  
    private ArrayList<Token> tokens;  
    private int pos = 0;  
  
    public Token peek() {  
        // 到末尾返回 null, 否则返回当前位置的 Token  
    }  
  
    public Token next() {  
        // 返回当前 Token, pos++  
    }  
}
```

实现提示

- `peek()`：检查 `pos >= tokens.size()` → 返回 `null`；否则返回 `tokens.get(pos)`
- `next()`：和 `peek()` 类似，但返回后要让 `pos++`
- 这两个方法各只需 2-3 行代码

这是整个 Project 03 中最简单的部分——先实现它，确保测试基础设施能跑通。

Project 03 — Parser 四个方法

四个方法的分工

<code>parse(tokens)</code>	← 入口: 创建 <code>TokenReadStream</code> , 调用 <code>parseForm</code>
↓	
<code>parseForm(rdr)</code>	← 决策: <code>LeftParen</code> → <code>parseList</code> , 否则 → <code>parseAtom</code>
↓	↓
<code>parseList(rdr)</code>	<code>parseAtom(rdr)</code>
消耗 (	消耗一个 <code>Token</code>
循环 <code>parseForm</code>	按类型创建 <code>LObject</code>
消耗)	
返回 <code>LList</code>	

关键: `parseForm` 和 `parseList` 相互递归——这就是"递归下降解析"。

Project 03 — Parser 伪代码

parseList — 核心递归逻辑

```
parseList(rdr):  
    rdr.next()           // 消耗 '('  
    children = []  
    while rdr.peek() != null && rdr.peek() 不是 RightParen:  
        children.add(parseForm(rdr))    // 递归调用!  
    rdr.next()           // 消耗 ')'  
    return new LList(children)
```

parseAtom — 将 Token 转为 AST 节点

```
parseAtom(rdr):  
    token = rdr.next()  
    if token 是 IntegerToken → return new LInteger(token.value)  
    if token 是 StringToken → return new LString(token.value)  
    if token 是 SymbolToken → return new LSymbol(token.name)
```

`parseList` 的 `while` 循环每轮调用 `parseForm`，`parseForm` 可能又调用 `parseList` ——这就是递归。

Project 03 — 解析示例(1/2)

输入: "(+ (* 2 3) 4)"

Token: [(+ (* 2 3) 4)]

```
parseForm → peek 是 ( → parseList
  next: 消耗 (
    parseForm → peek 是 + → parseAtom → LSymbol("+")
    parseForm → peek 是 ( → parseList      ← 递归进入内层
      next: 消耗 (
        parseForm → parseAtom → LSymbol("*")
        parseForm → parseAtom → LInteger(2)
        parseForm → parseAtom → LInteger(3)
        next: 消耗 )
        → LList[*, 2, 3]                    ← 内层返回
      parseForm → parseAtom → LInteger(4)
      next: 消耗 )
      → LList[+, LList[*, 2, 3], 4]        ← 外层返回
```

Project 03 — 解析示例(2/2)

最终 AST 结构

```
LList
├─ LSymbol("+")
├─ LList
│  └─ LSymbol("*")
│     └─ LInteger(2)
│        └─ LInteger(3)
└─ LInteger(4)
```

简单示例对比

"42"	→ LInteger(42)	单个原子
"(+ 1 2)"	→ LList[+, 1, 2]	一层列表
"(+ (* 2 3) 4)"	→ LList[+, LList[* , 2, 3], 4]	嵌套列表

规律：每遇到 (就递归进入下一层，遇到) 就返回上一层——层次自然形成。

Project 03 — 扩展任务

扩展 1: `astToString(LObject ast)` — AST 转字符串

AST 节点	输出
<code>LInteger(42)</code>	<code>"42"</code>
<code>LString("hi")</code>	<code>"\"hi\""</code>
<code>LSymbol("+")</code>	<code>"+"</code>
<code>LList([])</code>	<code>"()"</code>
<code>LList([+, 1, 2])</code>	<code>"(+ 1 2)"</code>

扩展 2: `countAtoms(LObject ast)` — 统计原子节点数

- 原子节点 (`LInteger` / `LString` / `LSymbol`) → 计 1
- `LList` → 各子节点 `countAtoms` 之和

```
countAtoms(parse(lex("(+ 1 2)")))           // → 3  ("+", 1, 2)
countAtoms(parse(lex("(+ (* 2 3) 4)")))      // → 5  ("+", "*", 2, 3, 4)
```

提示：两个扩展都是递归——对 `LList` 的每个子节点递归调用。

Project 03 — 提交要求

需实现的文件

文件	内容
TokenReadStream.java	peek() + next()
Parser.java	parse + parseForm + parseList + parseAtom + 两个扩展

运行测试

```
cd 03-ast
mvn test
```

14 个公开测试全部通过 = 完成

截止时间

截止 下次课开始前（一周内）

与 Lexer 的关系

- 03-AST 项目**已包含** Lexer 的完整实现
- 你只需实现 Parser 部分
- 测试会调用 `lex()` → `parse()` 的完整流水线

AI 使用说明

允许用 AI 辅助理解递归逻辑
不允许直接粘贴 AI 生成的完整解答

PART 3 · AI4Coding

DevOps 开发工具链 · 从代码到线上

什么是 DevOps?

DevOps = 开发 (Dev) + 运维 (Ops) + 文化/工具/实践
打破开发团队与运维团队之间的壁垒，让软件交付更快、更稳定。

传统模式

开发写完代码 → 扔给运维 → 运维手动部署 → 出问题互相推责

现代 DevOps 模式

代码提交 → 自动测试 → 自动构建 → 灰度发布 → 监控反馈 → 持续改进
↑ _____ 持续循环 (Infinity Loop) _____ ↑

DevOps 三大原则

原则	含义	实践
自动化	重复工作自动完成	CI/CD 流水线、自动测试、自动部署
测量	量化性能指标	错误率、响应时间、可用性监控
共享	跨团队协作与反馈	代码审查、故障复盘、知识沉淀

Git 版本控制 — 四个区域

工作区 (Working Directory)

↓ `git add`

暂存区 (Staging Area)

← "准备提交"的快照

↓ `git commit`

本地仓库 (Local Repository)

← 完整版本历史, 在 `.git` 目录

↓ `git push`

远程仓库 (Remote Repository)

← GitHub 上的副本, 团队共享

↑ `git pull` / `git fetch`

类比

- **工作区** = 你的桌面草稿纸
- **暂存区** = 你决定要装进信封的内容
- **本地仓库** = 你自己的档案柜
- **远程仓库** = 办公室公用的云端文件柜

大作业的 `mvn test` 在本地运行; 提交到 SEECODER 后在远程服务器上运行——这就是 CI 的雏形。

Git — 6 个最常用命令

1. 克隆远程仓库到本地

```
git clone https://github.com/username/repo.git
```

2. 查看当前状态（哪些文件改了、哪些暂存了）

```
git status
```

3. 将文件添加到暂存区

```
git add Lexer.java          # 指定文件
```

```
git add .                   # 当前目录全部
```

4. 提交到本地仓库

```
git commit -m "feat: 实现负整数解析"
```

5. 推送到远程仓库

```
git push origin main
```

6. 从远程拉取最新代码

```
git pull origin main
```

Git — Commit Message 规范

格式: `<type>: <subject>`

Type	含义	示例
feat	新功能	feat: 实现 lex() 主循环
fix	Bug 修复	fix: 修复负整数解析遗漏
test	测试	test: 添加字符串转义测试
refactor	重构	refactor: 提取 readInteger 辅助函数
docs	文档	docs: 更新 README
chore	配置	chore: 升级 JUnit 版本

```
# 好的 commit message
git commit -m "feat: 实现 parseList 递归解析"
git commit -m "fix: 修复空列表解析时的空指针异常"

# 不好的 commit message
git commit -m "update"           # 太模糊
git commit -m "fix bugs"         # 没有具体说明
```

习惯：每次 commit 只做一件事，写清楚做了什么。

CI/CD 流水线

CI (Continuous Integration): 每次提交自动测试

CD (Continuous Deployment): 测试通过后自动部署

开发者 `git push`

↓

自动触发测试 (`mvn test`)

↓

通过?

└ 是 → 自动构建 → 部署到测试环境 → 部署到生产环境

└ 否 → 通知开发者, 暂停部署

这和大作业有什么关系?

你在本地 `mvn test` → 你的本地 CI

提交到 SEECODER → 远程 CI (自动跑明测试 + 暗测试)

SEECODER 显示通过率 → 自动化测试结果

你已经在做 CI 了! 只是还没有 CD (自动部署)。

完整的现代开发工作流

1. 本地开发：编写代码
- ↓
2. `git add + git commit` (本地版本管理)
- ↓
3. `git push` (推送到远程)
- ↓
4. CI 自动运行 `mvn test` (GitHub Actions / SEECODER)
- ↓
5. Code Review (团队审查代码)
- ↓
6. 合并到 main 分支 → CD 自动部署到线上

工具链 — 你的对应实践

环节	专业工具	你现在的实践
编码	Cursor / IntelliJ	IDE 写代码
版本控制	Git + GitHub	<code>git add/commit/push</code>
自动测试	GitHub Actions	SEECODER <code>mvn test</code>
部署	Vercel / Docker	(后续课程)

AI 时代的 DevOps 演变

旧时代（2010–2019）

- 手动部署、手动测试
- 运维和开发分离
- 发布频率：季度或月度

AI 增强时代（2020 之后）

- **AI 驱动代码生成** → 更快的编码速度
- **自动化测试覆盖** → AI 生成测试用例
- **智能代码审查** → AI 识别潜在缺陷
- **发布频率** → 日度甚至小时级

你的 AI + DevOps 实践建议

1. **用 AI 辅助写代码** → Cursor / Copilot
2. **用 Git 管理版本** → 每个功能一个 commit
3. **用自动测试验证** → `mvn test` 养成习惯
4. **用 AI 审查代码** →

"请审查以下 Java 方法的代码质量：

1. 变量命名是否清晰？
2. 是否有可以简化的逻辑？
3. 是否存在边界条件遗漏？"

核心思维：AI 帮你写得快，DevOps 帮你改得稳——两者结合才是现代工程师的工作方式。

课后作业

必做

1. 完成 Project 03 AST

提交到 SEECODER-Coder, 截止下次课前。

实现 `TokenReadStream` + `Parser` 四个方法 + 两个扩展

2. 完成 Lab04 练习

4 道数组与 `ArrayList` 填空题

3. Git 实践

用 `git init` 管理你的 Project 03 代码,
至少提交 3 次有意义的 commit

选做

- 在 GitHub 上创建仓库, 将 Project 03 推送上去
- 尝试用 AI 解释 `parseList` 的递归逻辑
- 阅读 `LList.java` 源码, 理解 `getContent()` 为什么返回不可变列表

预习

下周内容: 类与对象 (OOP 入门)、继承

带着问题来:

Token 的类层次 `Token` → `IntegerToken` / `SymbolToken` / ...

AST 的类层次 `LObject` → `LInteger` / `LList` / ...

为什么要这样设计? 继承带来了什么好处?

当堂练习

Lab04 · 独立完成 · 30 分钟

Lab04 练习总览

#	名称	难度	考察重点	填空数
01	数组工具箱 ArrayTools	中等	数组遍历、查找、反转、截取	5
02	列表收集器 ListCollector	中等	ArrayList 增删改查、自动装箱	7
03	数据过滤器 DataFilter	较难	数组与ArrayList互转、去重、交集、归并	9
04	词频统计器 WordFrequency	较难	ArrayList<String>、.equals()、编排模式	10

运行方式

```
cd lab04/01-array-tools
mvn test # 运行全部测试
mvn test -Dtest=ArrayToolsTest # 只运行明测试
```

目标：至少完成前 2 题。第 3-4 题作为挑战，课后完成也可以。

练习 1 — 数组工具箱（中等）

15 min 实现 `ArrayTools` 类的 5 个方法，填写 5 个空：

```
public static int sum(int[] arr)           // 求和
public static boolean contains(int[] arr, int target) // 是否包含
public static int indexOf(int[] arr, int target) // 查找下标
public static int[] reverse(int[] arr)      // 反转（新数组）
public static int[] copyRange(int[] arr, int from, int to) // 截取子数组
```

关键填空

编号	方法	提示
填空1	<code>sum</code>	<code>sum += ?</code> （把 0 换成 <code>arr[i]</code> ）
填空2	<code>contains</code>	<code>arr[i] == target</code>
填空3	<code>indexOf</code>	找到时返回 <code>i</code>
填空4	<code>reverse</code>	<code>arr[arr.length - 1 - i]</code>
填空5	<code>copyRange</code>	<code>arr[from + i]</code>

练习 2 — 列表收集器（中等）

15 min 实现 `ListCollector` 类的 5 个方法，填写 7 个空：

```
public static ArrayList<Integer> fromArray(int[] arr)           // 数组→列表
public static int[] toArray(ArrayList<Integer> list)           // 列表→数组
public static boolean addUnique(ArrayList<Integer> list, int val) // 不重复添加
public static int removeAll(ArrayList<Integer> list, int val)   // 删除所有匹配
public static ArrayList<Integer> filterRange(..., int min, int max) // 范围筛选
```

注意事项

- `removeAll` 使用反向遍历（代码框架已提供反向循环）
- `addUnique` 先用 `list.contains(value)` 检查
- `filterRange` 判断 `n >= min && n <= max`

重点体会 `ArrayList` 与 数组 API 的差异：`list.get(i)` vs `arr[i]`

练习 3 — 数据过滤器：去重（较难）

挑战 实现 `DataFilter` 类的 5 个方法，填写 9 个空。

核心思路：用 ArrayList 做中间容器

```
// 不知道结果数组多长? → 先用 ArrayList 收集, 最后转数组
public static int[] removeDuplicates(int[] arr) {
    ArrayList<Integer> temp = new ArrayList<>();
    for (int i = 0; i < arr.length; i++) {
        if (!temp.contains(arr[i])) { // 填空3: 去重条件
            temp.add(arr[i]);         // 填空4: 添加
        }
    }
    return toArray(temp);           // 复用转换方法
}
```

这道题是“数组与ArrayList桥梁”——不知道结果长度时，先用 ArrayList 再转数组。

练习 3 — 数据过滤器：归并（较难）

mergeSorted — 双指针归并

```
a = [1, 3, 5]    i→  
b = [2, 4, 6]    j→  
result = []
```

比较 a[i] 和 b[j], 取较小的放入 result, 移动对应指针
→ [1, 2, 3, 4, 5, 6]

关键填空

编号	方法	提示
填空3	removeDuplicates	!temp.contains(arr[i])
填空5-6	intersection	bList.contains(a[i]) + 去重
填空7-9	mergeSorted	a[i] <= b[j] 比较 + 取元素

intersection（交集）的思路和 removeDuplicates 类似：遍历 a，检查 b 是否包含，同时去重。

练习 4 — 词频统计器（较难）

挑战 实现 `WordFrequency` 类的 5 个方法，填写 10 个空：

```
ArrayList<String> split(String text)           // 分词
int countWord(ArrayList<String> words, String t) // 词频统计
ArrayList<String> uniqueWords(...)           // 去重词列表
String mostFrequent(ArrayList<String> words)  // 最高频词
String buildReport(String text)              // 编排函数
```

编排模式再现（和 Lab03 的 `buildReport` 一样）

```
public static String buildReport(String text) {
    ArrayList<String> words = split(text);           // 填空8
    ArrayList<String> unique = uniqueWords(words);   // 填空9
    String top = mostFrequent(words);                 // 填空10
    return String.format("词数:%d 不同词数:%d 最高频词:%s",
        words.size(), unique.size(), top);
}
```

buildReport 只做编排，不做计算——这是单一职责原则的体现。

附录

JUnit 测试 · 提交规范 · 常见错误

附录 — 本周知识点速查

知识点	关键语法	注意事项
import	<code>import java.util.ArrayList;</code>	<code>java.lang.*</code> 自动导入
数组声明	<code>int[] arr = new int[5];</code>	长度固定，越界抛异常
数组长度	<code>arr.length</code>	属性，不是方法，没有 <code>()</code>
ArrayList 创建	<code>new ArrayList<Integer>()</code>	泛型只能用包装类
ArrayList 长度	<code>list.size()</code>	方法，有 <code>()</code>
字符串比较	<code>s1.equals(s2)</code>	永远不用 <code>==</code>
自动装箱	<code>list.add(42)</code>	<code>int</code> → <code>Integer</code> 自动转换
删除陷阱	<code>list.remove(3)</code>	删下标！要删值用 <code>Integer.valueOf(3)</code>
instanceof	<code>t instanceof IntegerToken</code>	判断对象类型
强制转型	<code>((IntegerToken) t).value</code>	先 instanceof 再转型

附录 一 运行测试与常见错误

运行测试

```
cd lab04/01-array-tools
mvn test # 运行全部测试
mvn test -Dtest=ArrayToolsTest # 只运行明测试
mvn test -Dtest=ArrayToolsTest#testSum_典型 # 运行单个测试
```

常见错误排查

错误信息	原因	修正
ArrayIndexOutOfBoundsException	数组越界	检查循环边界
NullPointerException	ArrayList 未初始化 / 方法返回 null	确保 new ArrayList<>()
AssertionError	输出不符合预期	检查格式、返回值
ClassCastException	错误的类型转换	先 instanceof 再转型
编译错误: cannot find symbol	缺少 import	添加 import java.util.ArrayList;

第四次编程实践课

软件工程与计算 I · 2026 · 南京大学

感谢聆听 · 有问题随时提问